

Revised Notebook:

https://colab.research.google.com/drive/1sRW_V5RSs1EAKMIGhfZebNEqGrzpViGt?usp=sharing

Introduction

When I first completed this portion of the homework, I was surprised by how short it was. Compared to other coding assignments in the course, this one contained only two subproblems, and each essentially required just a single line of code to complete (since each subproblem involved constructing a simple kernel filter).

Because of this, I decided to expand on the original homework: I added an additional problem, improved the formatting, and updated the code to follow both the Python Style Guide and the Google Python Style Guide.

Improvements

To begin, we have the introduction paragraph. LLMs are particularly good at rephrasing and improving such paragraphs when provided with adequate context and clear instructions. Below is the prompt we used to instruct the LLM to refine the paragraph according to the constraints specified in the assignment:

Improve the clarity, structure, and precision of the following paragraph. The text is meant to explain part of a deep learning course coding assignment, so rewrite it to be:

- technically accurate
- easy to follow for students
- concise but complete
- consistent in terminology
- logically organized (step-by-step when needed)

Do **not** change the technical meaning.

Do **not** add extra content unless necessary for clarity.

We also revised the initial setup cell, which imports the necessary packages, to improve organization, readability, and adherence to Python style guidelines.

Main changes include:

- Added descriptive docstrings to all functions
 - Docstrings improve readability and help others understand purpose, arguments, and return values
 - Reference: [PEP 257 Docstring Conventions](#)

Similarly, we also asked ChatGPT-5 to generate a transition paragraph for the section that loads the fox image. The difference in this case is that we used the same refinement prompt as above to have GPT-5 produce an improved version of the paragraph. We then manually implemented the actual image-loading logic—placing it into a helper function and adding appropriate type annotations.

Main changes include:

- Introduced type annotations (`np.ndarray`, `int`)
 - Improves clarity and enables static analysis and linting
 - Reference: [PEP 484 Type Hints](#)
- Removed unnecessary conversion to `torch.tensor` inside image display
 - Unused framework conversions violate simplicity principle
 - Reference: [PEP 20 \(Zen of Python\)](#)

For the box filter description, I felt the original explanation was overly long and not sufficiently concise. To address this, I used the same refinement prompt with ChatGPT to generate a clearer and more streamlined description. The improved version can be found in the corresponding section of the notebook.

For the actual code, main changes include:

- Introduced type annotations (`np.ndarray`, `int`)
 - Improves clarity and enables static analysis and linting
 - Reference: [PEP 484 Type Hints](#)
- Replaced implicit globals with explicit parameters and returns
 - Reduces hidden state and side effects
 - Reference: [Google Python Style Guide \(Functions and Side Effects\)](#)
- Ensured kernels explicitly use `dtype=float`
 - Matches expected numeric precision for convolution
- Added `mode='reflect'` when calling `ndimage.convolve`
 - Reflect padding matches typical CNN boundary behavior
- Used list annotations (`list[np.ndarray]`)
 - Improves readability and supports static checking
 - Reference: [PEP 585 – Type Hinting Generics In Standard Collections](#)

We also updated the Laplacian kernel section using the same refinement logic described earlier; therefore, we will not repeat those details here in order to keep the report concise.

Additionally, we introduced a new problem focused on Sobel edge detection. I felt that the original assignment was too short, and since Sobel filtering is another widely used edge detection algorithm, it made sense to include it as an extension. In this new section, students are asked to implement the Sobel kernels themselves, following the same style and structure as the previous problems.

For future iterations of the course, staff or students could expand on this foundation by adding more advanced edge detection topics—such as the Canny Edge Detector—and perhaps include a written question prompting students to compare the differences they observe between these techniques.